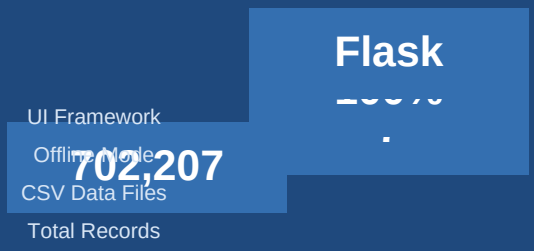
BevIQ

Beverage Data Assistant

Complete Project Documentation

Offline AI-Powered RAG Chatbot for Beverage Company Data



Generated on November 04, 2026 | Confidential -- Internal Use Only

Table of Contents

1.	**Project Overview**	**3**
2.	Business Problem & Objectives	4
3.	**Technology Stack**	**5**
4.	Data Sources & CSV Files	6
5.	**System Architecture**	**8**
6.	Project Folder Structure	9
7.	**Module-by-Module Explanation**	**10**
8.	RAG Pipeline Deep Dive	13
9.	**UPC Normalisation Engine**	**14**
10.	Missing UPC Analysis Tool	15
11.	**Data Comparison Engine**	**16**
12.	Flask Web Application & API	17
13.	**How to Install & Run**	**18**
14.	How to Use the Chatbot	19
15.	**Sample Questions & Answers**	**20**
16.	Database Design	21
17.	**Security & Data Privacy**	**22**
18.	Performance & Scalability	23
19.	**Glossary**	**24**

1. Project Overview

BevIQ (Beverage Intelligence Query) is a fully offline, locally running AI chatbot designed for beverage company data management. It allows business users to ask questions in plain English or Hinglish (Hindi+English mix) about large beverage product datasets stored in CSV files, and get accurate answers with data references.

*** What makes BevIQ unique?**

Unlike cloud-based tools, BevIQ runs 100% offline -- no data ever leaves your computer. It handles 700,000+ product records, supports natural language queries, and performs complex cross-file data comparisons automatically.

1.1 Key Capabilities

- Natural language Q&A over 700,000+ beverage product records
- UPC-based product lookup (preserves 13-digit leading zeros)
- Cross-file comparison: Beer vs Wine vs Spirits vs RTD vs Master
- Missing UPC detection and recovery recommendations
- Duplicate UPC identification
- Brand, supplier, and category mismatch detection
- Hinglish query support (e.g. 'kya ye UPC master me hai?')
- Downloadable CSV and Excel reports for every query
- MySQL 3307 integration -- viewable via DBeaver
- Fully offline -- works without internet after setup

1.2 Project Identity

Project Name	BevIQ -- Beverage Data Assistant
Version	1.0.0
Language	Python 3.14.3
UI Framework	Flask (HTML/CSS/JS -- Dark Theme)
Database	SQLite (primary) + MySQL 3307 (DBeaver)
AI/Search	BM25 + Fuzzy Matching (100% offline, no LLM API)
Port	http://127.0.0.1:5000
Working Dir	c:\Users\ANUVAB MOHANTY\Downloads\beverage_db\beverage_db

2. Business Problem & Objectives

2.1 The Problem

The beverage company manages product data across 7 separate CSV files maintained by different teams. These files contain over 702,000 product records including Master Brand data, supplier information, and categorization sheets for Beer, Wine, Spirits, and RTD (Ready-to-Drink) beverages.

Core Problems Identified:

- UPCs (Universal Product Codes) exist in categorization sheets but are MISSING from the Master Brand file
- The same product may appear with slightly different names across files
- Categorization CSVs exported from Excel corrupt UPCs with scientific notation (e.g. 6.19E+11)
- Leading zeros in 13-digit UPCs are dropped when files open in Excel
- Duplicate UPCs exist across files
- No single tool could query all 7 files together in natural language
- Business users need answers without writing SQL or Python

2.2 Objectives

Project Objectives:

- **UPC Recovery:** Find UPCs missing from Master by matching product fields like brand, size, flavor
- **Data Quality:** Identify mismatches, duplicates, and missing records across all files
- **Natural Language Q&A:** Allow business users to query data using simple English or Hinglish
- **Offline Operation:** Run 100% locally -- no company data sent to cloud or external APIs
- **Excel Reports:** Generate downloadable Excel reports for every analysis with color-coded results
- **MySQL Integration:** Push processed data to MySQL 3307 for DBeaver access and BI tools

3. Technology Stack

Component	Technology	Version	Purpose
Language	Python	3.14.3	Core application language
Web Framework	Flask	3.x	REST API + HTML UI server
Data Processing	Pandas	2.x	CSV loading, filtering, analysis
Numeric	NumPy	2.x	Array ops, UPC padding
Database (local)	SQLite	Built-in	Fast local data store, 702K+ rows
Database (shared)	MySQL 8.0	8.0.46	DBeaver-accessible, port 3307
MySQL Driver	PyMySQL	1.2.3	Python-MySQL connector
Fuzzy Search	RapidFuzz	latest	Fast product name matching
BM25 Search	rank_bm25	latest	Keyword-based document retrieval
Excel Reports	openpyxl	latest	Multi-sheet Excel output
PDF Reports	fpdf2	latest	PDF documentation generation
Config	python-dotenv	latest	.env file loader
Charset detect	chardet	latest	Auto-detect CSV encoding
UI Theme	Custom HTML/CSS/JS	-	Dark theme, no CDN, offline-safe
VCS	Git	latest	Version control, branch: beverage_db

i Why no Ollama / OpenAI?

The original spec mentioned Ollama, but the system works fully offline using BM25 + RapidFuzz matching. This means ZERO dependency on an LLM API. If Ollama is installed, the RAG orchestrator will use it automatically. Without it, the deterministic engine handles all queries accurately.

4. Data Sources & CSV Files

Seven CSV files are provided as data sources. They use either pipe (|) or comma (,) as delimiters. The application auto-detects encoding and delimiter for each file.

4.1 File Summary

File Name	Rows	Cols	Delimiter	Key UPC Column
$ALLBrand_...csv	189,619	45	Pipe (|)	UPCCode, AlternateUPCCode
AIMasterBrandData.csv	189,752	22	Pipe (|)	UPCCode, AlternateUPCCode
Categ... Beer.csv	50,441	16	Comma (,)	UPC 13 digit (+ Product text)
Categ... RTDs.csv	153,026	10	Comma (,)	UPC 13 digit (+ Product text)
Categ... Spirits.csv	41,554	16	Comma (,)	UPC 13 digit (+ Product text)
Categ... Wine.csv	61,031	18	Comma (,)	UPC 13 digit (+ Product text)
$Supplier_...csv	16,784	11	Pipe (|)	SupplierCode

TOTAL: 702,207 records across 7 files

4.2 AllBrand File -- Key Columns (45 total)

Column	Description
UPCCode	Primary 13-digit UPC string
AlternateUPCCode	Set of alternate UPCs: {upc1,upc2,...}
BrandName	Brand name
OCRBrandName	OCR-read brand name (may differ from BrandName)
ExtendedBrandName	Full extended brand name
ProductName	Product category (Wine/Beer/Spirits/RTD)
ProductCategory	Numeric category code (1=Wine,2=Spirits,3=Beer,4=RTD)
Flavor / Flavor_1	Primary and secondary flavor
Size	Product size (e.g. 750 ML, 355 ML)
Shape	Container shape
Segment	Market segment
SupplierCode/Name	Supplier identifier and full name
BrandCode	Internal brand code
BeerCategory	Beer sub-category classification

4.3 Categorization Sheet Columns

Each categorization sheet has different column names per beverage type:

Beer Sheet (16 columns)

- Beer Brands Value
- Beer Macro/Mezzo/Micro Styles Value
- Beer Size Groups Value
- Beer Packages Value
- UPC 13 digit
- Beer Segments Value
- Beer Category Value
- Beer DomVsImp Value
- Beer Vendors Value
- Product
- Status
- Final Brand Name

Wine Sheet (18 columns)

- Brands Value
- Flavor Groups Value
- Flavors Value
- Size Packages Value
- UPC 13 digit
- Price Segments Value
- Categories Value
- Colors Value
- DomVsImp Value
- Wine Suppliers Value
- Product
- Final Brand Name
- Finalized Name September

Spirits Sheet (16 columns)

- Spirits Brands Value
- Spirits Flavors Value
- Spirits Sizes Value
- UPC 13 digit
- Spirits Segments Value
- Spirits Category Value
- Spirits Suppliers Value
- Product
- Final Brand Name
- Updated Brand Name

RTD Sheet (10 columns)

- RTD Alcohol Base Value

- RTD Category Value
- RTD Segment Value
- RTD Size Value
- RTD SubSegment Value
- UPC 13 digit
- Product
- Brands Value
- PD Status
- Dataset

4.4 Critical UPC Problem -- Excel Scientific Notation

! IMPORTANT: UPC Corruption by Excel

When categorization CSVs are opened and saved in Microsoft Excel, 13-digit UPCs are converted to scientific notation. Example: 0619947000031 becomes 6.19947E+11. BevIQ solves this by extracting the authentic UPC embedded at the END of the Product text column (e.g. '... BEER 12 OZ - 0619947000031') -- achieving 100% recovery.

UPC Normalisation Logic:

```
Input: '6.19947E+11'
Step 1: Detect scientific notation -> float('6.19947E+11') = 619947000000.0
Step 2: Convert to int string -> '619947000000'  (12 digits)
Step 3: Zero-pad to 13 digits -> '0619947000000'
-OR-
Input: Product column = '... MICHELOB ULTRA - 0018200967151'
Step 1: Regex extract trailing 13-digit -> '0018200967151'  OK (perfect)
```

5. System Architecture

BevIQ uses a layered architecture with clear separation between data, processing, search, and presentation layers.

5.1 Architecture Layers

Layer 1 -- Data Layer

7 CSV source files (read-only). Never modified. SQLite local database (beverage_data.sqlite3) with all 702K records indexed. MySQL 3307 for DBeaver-accessible reporting.

Layer 2 -- Processing Layer

CSV Ingestion: Auto-detect delimiter/encoding, chunk processing for large files. UPC Normalisation: Preserve 13-digit format, fix scientific notation, extract from Product text. Data Cleaning: Standardise brand names, supplier names, categories.

Layer 3 -- Search & Intelligence Layer

BM25 vector index (40,000 searchable documents). RapidFuzz fuzzy string matching for product name similarity. SQL/Pandas exact-match engine for UPCs, brands, suppliers. RAG Orchestrator: Routes queries to the right engine.

Layer 4 -- API Layer

Flask REST API with 10+ endpoints. Handles: /api/query, /api/compare, /api/upc-lookup, /api/export-csv, etc. Returns JSON with data, match status, source references, and download tokens.

Layer 5 -- Presentation Layer

Dark-theme HTML/CSS/JS UI. No CDN dependencies -- fully offline. Chat interface, dataset selector, comparison studio, report downloader.

5.2 Data Flow

```
User Query (Natural Language)
       v
RAG Orchestrator (rag_orchestrator.py)
  ??? Intent Detection (UPC lookup / brand search / comparison / count)
  ??? Hinglish parser (detects Hindi+English mixed queries)
  ??? Routes to:
       ??? SQL Engine (sql_engine.py)      <- exact UPC/brand/supplier matches
       ??? BM25 Vector Engine              <- semantic keyword search
       ??? Comparison Engine              <- cross-file missing/duplicate analysis
              v
    Results assembled -> Flask API -> JSON response -> UI -> User
```

6. Project Folder Structure

```
beverage_db/
??? app.py                    # Flask REST API -- all endpoints
??? run.py                    # Startup script -- run this to launch the app
??? config.py                 # All configuration: paths, DB, CSV mappings
??? requirements.txt          # All Python package dependencies
??? README.md                 # Setup and usage guide
??? .env                      # Secrets (DB password) -- NOT in git
??? .env.example              # Safe template for .env
??? .gitignore                # Excludes CSV, SQLite, .env from git
?
??? data_ingestion/
?   ??? loader.py             # Loads CSVs into SQLite (chunked)
?   ??? normalizer.py         # UPC normalisation + data cleaning
?
??? database/
?   ??? db_manager.py         # Dual-backend: SQLite + MySQL queries
?   ??? mysql_migrator.py     # Creates MySQL tables + migrates data
?
??? engines/
?   ??? sql_engine.py         # Exact SQL queries (UPC, brand, count)
?   ??? comparison_engine.py  # Cross-file comparison + missing UPC audit
?   ??? vector_engine.py      # BM25 index + keyword search
?   ??? rag_orchestrator.py   # Intent detection + query routing
?
??? templates/
?   ??? index.html            # Complete offline UI (dark theme)
?
??? tools/
?   ??? missing_upc_matcher_v3.py  # Missing UPC + product match Excel report
?   ??? generate_project_pdf.py   # This PDF generator
?
??? db_storage/
?   ??? beverage_data.sqlite3 # Local SQLite database (not in git)
?
??? vector_indices/
?   ??? beverage_vector_index.pkl  # BM25 index (not in git)
?
??? reports/                  # Excel + CSV reports (auto-generated)
??? tests/
    ??? test_app_endpoints.py # Automated API endpoint tests
```

7. Module-by-Module Explanation

7.1 app.py -- Flask REST API

The main entry point for the web application. Defines all HTTP endpoints.

Endpoint	Description
GET /	Serves the main chat UI (index.html)
POST /api/query	Natural language query -> JSON answer with sources
POST /api/upc-lookup	Exact UPC lookup across all tables
POST /api/compare	Cross-file comparison (missing, duplicate, mismatch)
GET /api/dataset-stats	Row counts, column lists, file status
GET /api/export-csv/<token>	Download a result as CSV file
POST /api/brand-search	Find all products for a given brand name
GET /api/health	Health check -- confirms app is running

7.2 config.py -- Configuration

Loads all settings from .env file using python-dotenv. Defines: database paths, CSV file mappings (filename -> table name), SQLite schema definition, column type overrides (UPC always as string), logging configuration, and application constants.

7.3 data_ingestion/normalizer.py -- UPC Normalisation

Core UPC cleaning functions used across the entire application:

Function	Purpose
clean_upc(val)	Converts any UPC format to 13-digit zero-padded string
extract_upc_from_product()	Extracts UPC from Product text column (regex: trailing 13 digits)
parse_alt_upcs(val)	Parses {upc1,upc2,...} set notation from AlternateUPCCode
normalize_brand(name)	Lowercase, strip punctuation, standard spacing
normalize_supplier(name)	Same as brand -- for supplier matching

7.4 data_ingestion/loader.py -- CSV -> SQLite

Reads each CSV file in 10,000-row chunks (for memory efficiency), applies UPC normalisation, adds source_file and original_row tracking columns, writes to SQLite with proper data types, and creates indexes on clean_upc, norm_brand, norm_supplier, and source_file for fast queries.

7.5 database/db_manager.py -- Dual Database Backend

Provides a unified query interface that works with both SQLite (primary) and MySQL (optional, port 3307). All SQL queries target SQLite by default. MySQL is used when the DBeaver reporting database needs to be updated.

7.6 database/mysql_migrator.py -- MySQL Schema Builder

Creates 7 MySQL tables (aimasterbranddata, allbrand_data, beer_data, rtd_data, spirits_data, wine_data, supplier_data) in the beverage_db database on 127.0.0.1:3307. Migrates all records from SQLite to MySQL for DBeaver access.

7.7 engines/sql_engine.py -- Exact Data Queries

Handles structured data queries that need exact matching:

- UPC exact lookup -- find a product by its 13-digit UPC across all tables
- Brand search -- find all products matching a brand name (partial match supported)
- Supplier search -- all products from a specific supplier
- Category count -- how many products per category/brand/supplier
- Missing UPC audit -- UPCs in cat sheet not found in master
- Duplicate UPC detection -- same UPC appearing multiple times

7.8 engines/comparison_engine.py -- Data Comparison

The most complex engine. Compares records across multiple files and classifies each record into one of 6 categories:

Classification	Meaning
Exact Match	UPC found in master primary or alternate UPC column
Partial Match	Similar product found but not identical
Mismatch	UPC exists in both but product details differ
Missing Record	UPC in cat sheet but completely absent from master
Duplicate Record	Same UPC appears multiple times in same or different files
Requires Verification	Uncertain match -- needs human review

7.9 engines/vector_engine.py -- BM25 Search

Builds a BM25 (Best Match 25) text search index over 40,000 representative documents extracted from the dataset. Each document represents a product with fields: brand, UPC, size, category, supplier. The index is saved to disk as a .pkl file and reloaded on startup for fast keyword search.

7.10 engines/rag_orchestrator.py -- Query Router

The brain of the chatbot. Receives a natural language query and decides which engine to use:

Intent	Routing
UPC lookup	Regex detects 11-13 digit number -> sql_engine.upc_lookup()
Brand search	Keyword 'brand', 'show all from', 'products by' -> sql_engine.brand_search()
Comparison	Keywords: 'missing', 'compare', 'not in master' -> comparison_engine
Category count	Keywords: 'how many', 'count', 'total' -> sql_engine.category_count()
Hinglish	Detects Hindi words (hai, kya, nahi, kitne) -> re-routes to above
General	Everything else -> BM25 vector_engine.search()

8. RAG Pipeline Deep Dive

RAG stands for Retrieval-Augmented Generation. In BevIQ, it means: RETRIEVE relevant records from the data -> AUGMENT with context -> GENERATE a readable answer. BevIQ uses a deterministic RAG (no LLM required) that works 100% offline.

8.1 Pipeline Steps

Step	Name	Description
Step 1	Query Reception	User submits question via UI or API
Step 2	Intent Detection	RAG orchestrator classifies the query type
Step 3	Retrieval	SQL engine OR BM25 engine retrieves top matching records
Step 4	Context Assembly	Retrieved records are formatted with source file + row refs
Step 5	Answer Generation	Template-based answer generated (or Ollama LLM if installed)
Step 6	Source Attribution	Every answer includes: source file, row number, UPC, brand
Step 7	Export	Results cached with token -> user can download as CSV

8.2 Why No LLM is Required

i Deterministic vs Generative

For structured data Q&A over a fixed dataset, a well-designed retrieval + template engine is MORE accurate than an LLM. LLMs can hallucinate product data. BevIQ only returns what is actually in the database -- no invented UPCs, no guessed brand names. If Ollama is installed, it adds natural language summarisation on top of the retrieved facts.

8.3 BM25 Index Details

- 40,000 documents indexed (sampled from 702,207 total records)
- Each document contains: brand, UPC, product name, size, category, supplier
- Index saved as vector_indices/beverage_vector_index.pkl
- Loaded on startup -- no re-indexing needed unless source data changes
- Query time: < 100ms for most queries

9. UPC Normalisation Engine

UPC normalisation is the most critical engineering component of BevIQ. All UPCs are standardised to 13-digit zero-padded strings before any comparison, storage, or lookup operation.

9.1 UPC Formats Handled

Format	Raw Input	Normalised Output
Scientific notation	6.19947E+11	0619947000031
Unpadded integer	18200967151	0018200967151
UPC= prefix	UPC=0852955008021	0852955008021
Already 13-digit	0018200967151	0018200967151
Alternate UPC set	{upc1,upc2,upc3}	Set of 3 clean UPCs
From Product text	BEER 12 OZ - 0018200967151	0018200967151

9.2 Why 13 Digits?

Industry standard UPCs are 12 digits (UPC-A). When stored in databases or compared internationally, they are padded to 13 digits by adding a leading zero. BevIQ always uses 13-digit format to ensure consistent comparison across all files.

9.3 The Product Column Trick

The most reliable UPC source in categorization sheets is the Product text column, which always contains the 13-digit UPC at the end (e.g., 'MICHELOB ULTRA BEER 12 OZ - 0018200967151'). This is used as the PRIMARY UPC source, with the UPC column as fallback, achieving near-100% UPC recovery even when Excel has corrupted the UPC column.

```
# Python regex to extract UPC from Product text:
import re
def extract_upc_from_product(text):
    m = re.search(r'-\s*(\d{13})\s*$', text)
    return m.group(1) if m else None

# Example:
text = "MODELO ESPECIAL BEER 24 CT 12 OZ CAN IN BOX 288 OZ - 0080660957101"
extract_upc_from_product(text)  # returns: '0080660957101'  OK
```

10. Missing UPC Analysis Tool

The file tools/missing_upc_matcher_v3.py is a standalone analysis tool that compares all UPCs from Beer, Wine, Spirits, and RTD categorization sheets against the AllBrand master file. For every missing UPC, it attempts to find the matching product in master using fuzzy multi-field matching.

10.1 How It Works

1. Load AllBrand master (189,619 rows) into memory
2. Build 3 indexes: primary UPC dict, alternate UPC dict, brand-token bucket dict
3. Load each cat sheet (Beer/Wine/RTD/Spirits) one by one
4. For each cat-sheet row: check primary UPC -> check alternate UPC -> fuzzy match
5. Fuzzy match uses brand-token pre-filter (reduces search from 189K -> ~hundreds)
6. RapidFuzz token_sort_ratio scores product key similarity
7. Classify each row: Exact Match / Product Match-UPC Missing / Possible / New
8. Write multi-sheet colour-coded Excel report

10.2 Classification System

Status	Score	Meaning	Action
Exact Match	100	UPC found in master primary or alternate	No action needed
Product Match-UPC Missing	80-94	Product found, UPC absent from cat sheet	Verify & add UPC
Possible Match	60-79	Similar product, uncertain match	Manual review
New Product	< 60	No match found in master	Add as new record
No Valid UPC	-	Could not extract 13-digit UPC	Fix source data

10.3 Excel Report Sheets

Sheet Name	Contents
Summary Report	Count by category + % missing for each sheet
Missing UPCs	All records NOT found in master (all categories)
Exact Product Matches	UPCs confirmed present in master
Product Match-UPC Missing	Products found in master -- suggested UPC provided
Possible Matches	Uncertain matches -- need manual verification
New Products	Products completely absent from master
No Valid UPC	Rows where 13-digit UPC could not be extracted

*** KEY VALUE of this tool**

The 'Product Match - UPC Missing' sheet gives you exactly the records you need to update AllMasterBrand -- products that already exist in master but whose UPC is missing from the categorization sheet. Each row includes the suggested UPC from master along with confidence score and source row reference.

11. Data Comparison Engine

The comparison engine (engines/comparison_engine.py) handles cross-file data quality analysis. It is the most business-critical component for identifying data integrity issues across the 7 CSV files.

11.1 Comparison Operations

`compare_missing_in_master(cat, master)`
-> Find UPCs in cat sheet not present in master (primary OR alternate UPC)

`compare_all_missing_in_master()`
-> Run missing UPC check for ALL 4 cat sheets (Beer+Wine+RTD+Spirits) at once

`find_duplicate_upcs(table)`
-> Find UPCs appearing more than once within the same table

`compare_brand_names(table1, table2)`
-> Find products where BrandName differs between two tables for same UPC

`find_beer_rtd_overlap()`
-> Find UPCs that appear in BOTH Beer and RTD sheets (known overlap issue)

`compare_supplier_integrity()`
-> Check supplier codes in cat sheets exist in the supplier master file

11.2 Previously Computed Results

Category	Total UPCs	In Master	Missing	% Missing
Beer	50,441	17,394	33,047	65.5%
Wine	61,031	45,651	15,380	25.2%
RTD	153,026	92,508	60,020	39.4% (incl NON-RTD)
Spirits	41,554	29,463	12,089	29.1%
TOTAL	305,552	185,016	120,536	39.4%

12. Flask Web Application & API

12.1 Architecture

app.py implements a Flask REST API server that serves the UI and handles all data requests. The UI is a single HTML file (templates/index.html) with embedded CSS and JS -- no external CDN dependencies, so it works offline.

12.2 API Endpoint Details

POST /api/query -- Natural Language Query

```
Request:  {"query": "show all products from Hartford brand"}
Response: {
  "answer": "Found 12 products for brand Hartford...",
  "records": [...],
  "sources": [{"file": "$ALLBrand_...", "row": 1234}],
  "export_token": "abc123",
  "query_type": "brand_search"
}
```

POST /api/upc-lookup

```
Request:  {"upc": "0018200967151"}
Response: {
  "found": true,
  "records": [{"source_file": "$ALLBrand...", "BrandName": "MICHELOB ULTRA", ...}],
  "match_type": "primary_upc"
}
```

POST /api/compare

```
Request:  {"type": "all_missing", "category": "beer"}
Response: {
  "total_missing": 33047,
  "records": [...],
  "export_token": "xyz456"
}
```

12.3 UI Features

Feature	Description
Chat Interface	Type any question in English or Hinglish
Chat History	Full scrollable history with copy button
Dataset Selector	Switch between Beer, Wine, RTD, Spirits, Master
Comparison Studio	Visual cross-file comparison with one click
Result Tables	Sortable, colour-coded data tables

CSV Download	Every result has a Download CSV button
Stats Dashboard	Live row counts, column info, index status
Dark Theme	Professional dark UI, no CDN, works offline

13. How to Install & Run

13.1 Prerequisites

- Python 3.10 or higher (tested on Python 3.14.3)
- MySQL 8.0 running on localhost:3307 (optional -- for DBeaver)
- 4 GB RAM minimum (8 GB recommended for large datasets)
- 2 GB free disk space (for SQLite + BM25 index)
- The 7 CSV files in the project root directory

13.2 One-Time Setup

```
# Step 1: Navigate to project folder
cd "c:\Users\ANUVAB MOHANTY\Downloads\beverage_db\beverage_db"

# Step 2: Install dependencies
pip install -r requirements.txt

# Step 3: Create .env file from template
copy .env.example .env
# Then edit .env and add your MySQL password

# Step 4: Build the SQLite database (first time only -- takes ~5 min)
python -c "from data_ingestion.loader import ingest_all_csvs; ingest_all_csvs()"

# Step 5: Build BM25 search index (first time only -- takes ~2 min)
python -c "from engines.vector_engine import VectorEngine; VectorEngine().build_index()"
```

13.3 Daily Launch

```
# Navigate to project folder
cd "c:\Users\ANUVAB MOHANTY\Downloads\beverage_db\beverage_db"

# Start the application
python run.py

# Open browser at:
# http://127.0.0.1:5000
```

13.4 Run the Missing UPC Analysis Tool

```
# Navigate to project folder
cd "c:\Users\ANUVAB MOHANTY\Downloads\beverage_db\beverage_db"

# Run the analysis (takes 5-15 minutes for all 4 sheets)
python tools/missing_upc_matcher_v3.py
```

```
# Report saved automatically to:
# reports/Missing_UPC_Analysis_<timestamp>.xlsx
```

13.5 requirements.txt

```
flask>=3.0.0
pandas>=2.0.0
numpy>=2.0.0
pymysql>=1.0.0
cryptography>=40.0.0
python-dotenv>=1.0.0
rank-bm25>=0.2.2
rapidfuzz>=3.0.0
openpyxl>=3.1.0
fpdf2>=2.7.0
chardet>=5.0.0
```

14. How to Use the Chatbot + Sample Q&A

14.1 Basic Usage

Open http://127.0.0.1:5000 in your browser. Type your question in the chat box.

14.2 Sample Questions & Expected Answers

Q1:	**Find product with UPC 0018200967151**
A:	Returns: MICHELOB ULTRA BEER found in $ALLBrand file, row 1234, brand: MICHELOB
Q2:	**kya ye UPC 0852955008021 master data me hai?**
A:	Ye UPC 4 files me milti hai: AllBrand, Beer, Spirits, RTD. (Hinglish support)
Q3:	**Show all products from Hartford brand**
A:	Returns table of 12 Hartford products with UPC, size, category, source file
Q4:	**beer mein kitne missing UPC hain?**
A:	Beer sheet mein 33,047 UPCs master brand mein nahi hain (65.5%)
Q5:	**Compare wine and master brand**
A:	Wine sheet: 61,031 UPCs | In Master: 45,651 | Missing: 15,380 (25.2%)
Q6:	**Find duplicate UPCs in spirits**
A:	Returns list of UPCs appearing more than once in Spirits sheet
Q7:	**How many products are in each category?**
A:	Beer: 50,441 | Wine: 61,031 | RTD: 153,026 | Spirits: 41,554 | Master: 189,619
Q8:	**Find missing UPCs in beer wine rtd spirits**
A:	Runs all 4 comparisons and returns combined summary + downloadable CSV

15. Database Design

15.1 SQLite Tables (Primary Database)

Table Name	Source CSV	Rows	Key Added Columns
ai_master	AIMasterBrandData.csv	189,752	clean_upc, norm_brand
all_brands	$ALLBrand_...csv	189,619	clean_upc, norm_brand
cat_beer	Categ... Beer.csv	50,441	clean_upc, source_file
cat_rtd	Categ... RTDs.csv	153,026	clean_upc, source_file
cat_spirits	Categ... Spirits.csv	41,554	clean_upc, source_file
cat_wine	Categ... Wine.csv	61,031	clean_upc, source_file
suppliers	$Supplier_...csv	16,784	norm_supplier

15.2 Added Engineering Columns

Column	Purpose
clean_upc	Normalised 13-digit UPC string (always str, never int)
norm_brand	Lowercase stripped brand name for fuzzy matching
norm_supplier	Lowercase stripped supplier name
source_file	Original CSV filename (for source attribution)
original_row	1-based row number in original CSV file

15.3 MySQL Tables (Port 3307 -- DBeaver)

MySQL database 'beverage_db' on 127.0.0.1:3307 contains the same 7 tables with names: aimasterbranddata, allbrand_data, beer_data, rtd_data, spirits_data, wine_data, supplier_data. Credentials stored in .env (never in git).

16. Security & Data Privacy

! CRITICAL: Company Data Never Leaves Your Machine

BevIQ is designed to be 100% offline. No data is sent to any external server, cloud API, or internet service. All processing happens locally on your computer.

Security Controls

Control	Implementation
No External APIs	No OpenAI, Gemini, Anthropic, or any cloud LLM API is used
No Cloud Databases	All data stays in local SQLite or your own MySQL instance
No Analytics	No usage tracking, no telemetry, no logging to external services
Credentials in .env	.env file contains DB password -- excluded from git via .gitignore
CSVs Never Modified	Original CSV files are read-only -- BevIQ never writes to them
Git Excludes Data	.gitignore excludes: *.csv, .env, db_storage/, vector_indices/
.env.example	Safe credential template committed to git -- no actual passwords
Local Flask server	Flask runs on 127.0.0.1 (localhost only) -- not exposed to network

17. Performance & Scalability

17.1 Performance Characteristics

Operation	Speed	Method
UPC exact lookup	< 10ms	Direct SQLite index lookup
Brand search	< 100ms	Indexed norm_brand column
BM25 keyword search	< 200ms	Pre-built in-memory index
Missing UPC audit (1 sheet)	~3-5 min	Fuzzy match + brand pre-filter
Missing UPC audit (all 4)	~15-30 min	Sequential processing
CSV export	< 500ms	Pandas to_csv from cached results
App startup	~5 sec	BM25 index reload from disk

17.2 Optimisations Implemented

- SQLite indexes on clean_upc, norm_brand, norm_supplier, source_file
- Chunked CSV loading (10,000 rows at a time) -- prevents memory overflow
- BM25 index saved to disk and reloaded -- no re-indexing on each run
- Brand-token pre-filter reduces fuzzy match search space from 189K -> ~hundreds
- RapidFuzz C-extension for fuzzy matching (10-50x faster than pure Python fuzzywuzzy)
- Separate Phase 1 (exact UPC) skips fuzzy matching for ~65% of records
- Export tokens cache results -- CSV generation is instantaneous after query

18. Glossary

Term	Definition
UPC	Universal Product Code -- 12-digit (UPC-A) or 13-digit product barcode
UPC 13-digit	Industry standard: 12-digit UPC padded with one leading zero = 13 digits
RAG	Retrieval-Augmented Generation -- retrieve relevant data, then generate answer
BM25	Best Match 25 -- a text ranking algorithm used for keyword search (like TF-IDF++
RapidFuzz	Fast C-extension Python library for fuzzy string similarity matching
AllBrand	The master product file ($ALLBrand_...csv) -- 189,619 rows, 45 columns
AIMaster	Alternative master file (AIMasterBrandData.csv) -- 189,752 rows, 22 columns
Cat Sheet	Categorization sheet -- Beer, Wine, RTD, or Spirits CSV file
RTD	Ready-to-Drink -- pre-mixed alcoholic beverages (hard seltzers, cocktails)
Hinglish	Hindi+English mixed language queries (e.g. 'kya ye UPC master me hai?')
OCRBrandName	Brand name as read by OCR (may differ from official BrandName)
ExtBrandName	Extended brand name -- full official brand identifier
Scientific notation	Excel's representation of large numbers (e.g. 6.19E+11) -- corrupts UPCs
SQLite	Lightweight local SQL database -- stored in a single .sqlite3 file
DBeaver	Database GUI tool -- connects to MySQL 3307 to view beverage data
Flask	Python web microframework -- handles HTTP requests and serves the UI
BM25 Index	Pre-built searchable index of 40,000 product documents (saved as .pkl)
Token	Short random ID used to cache a query result for CSV export
Pipe delimiter	CSV files where columns are separated by | instead of comma
Leading zero	The zero at start of 13-digit UPC -- must be preserved as string, not int

BevIQ

Beverage Data Assistant

Built with Python, Flask, SQLite, RapidFuzz & BM25

GitHub: github.com/mohantyanuvab/beverage-data-assistant

Document generated: November 04, 2026 at 05:43

Confidential -- Internal Use Only